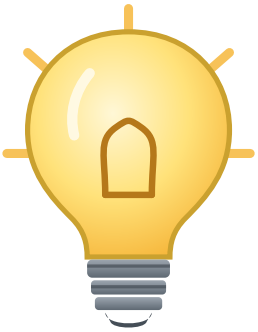
PROTOTYPER Ed Barlow / PC Setup | Ed Barlow · Web Cloud Studio



OPERATING ENVIRONMENT REFERENCE

Ed Barlow's PC Setup

If I document it, it will become accurate.

Ed Barlow Web Cloud Studio · 2026

Purpose

Document my development environment and tooling.

```
bash Prototyper/bin/build_documentation.sh                                  # → Master Documentat
bash Prototyper/bin/build_pdf.sh docs/whitepapers/ed_barlow_setup.md        # → ./ed_barlow_setup
```

Environment

Property	Value
Platform	Ubuntu 22.04 (Jammy) on WSL2, hosted under Windows
Shell	bash
GPU	NVIDIA RTX 4060 Laptop (8 GB); `nvidia-smi` works inside WSL
`$MYPROJECTS`	Root directory for all git repositories (`/mnt/c/Users/barlo/projects`)

The working interface is bash inside WSL. Windows-side applications (Docker Desktop, Ollama, Anaconda) are reached from bash through WSL interop, never through a PowerShell console.

Installed Tooling

System CLI (apt)

Tool	Use
`tree`	Directory structure at a glance.
`shellcheck`	Lint `bin/*.sh` before committing.

Other command-line utilities are listed under **Utilities** below.

Language toolchains

Tool	Use
`uv`	Virtual environments and dependency management in WSL.
`ruff`	Python linting and formatting.
`pytest`	Test runner; `bin/test.sh` wraps it and must pass before commit.
`playwright`	Headless Chromium — drives the white-paper → PDF render.
`node` (via `nvm`)	Node 20 toolchain.

Anaconda lives on the **Windows** side; `uv` is the WSL standard. They do not overlap.

SQLite and the CLI access pattern

SQLite is the default persistent store, reached directly with the `sqlite3` client:

```
sqlite3 data/app.db                                  # interactive shell
sqlite3 data/app.db '.tables'                         # list tables
sqlite3 data/app.db '.schema users'                   # inspect a table
sqlite3 -header -column data/app.db 'SELECT * FROM users LIMIT 20;'   # readable query output
sqlite3 data/app.db '.mode csv' '.import in.csv users'               # bulk load
```

`-header -column` is the standard pair for legible ad-hoc queries. Application code reaches the same database only through its typed access class — the `sqlite3` client is for inspection, not for production writes.

Docker

Docker Desktop runs on Windows; the `docker` CLI is exposed to bash through **WSL integration**. When integration is off, `docker` resolves to the Windows binary but the daemon is unreachable.

```
# 1. Launch Docker Desktop and enable WSL integration (GUI, one time):
#    Settings (gear) → Resources → WSL Integration →
#    enable "default WSL distro" and toggle Ubuntu → Apply & Restart.
# 2. Verify from bash:
docker version
docker run --rm hello-world

# Update Docker Desktop from bash (winget runs from bash, not PowerShell):
winget.exe upgrade --id Docker.DockerDesktop
# Or: launch the app and use Settings → Software Updates → Check for updates.
"/mnt/c/Program Files/Docker/Docker/Docker Desktop.exe" &
```

Local models (Ollama + GPU)

Ollama is installed on Windows and called from WSL through the `ollama` alias. The RTX 4060 (8 GB) runs 7–8B quantized models comfortably.

```
export OLLAMA_HOST=127.0.0.1:11434                   # in .bashrc
ollama serve &            # or start the Windows tray app
ollama pull llama3.1:8b
curl localhost:11434/api/tags                         # confirm the API is up
```

Platform tooling

Tool	Location	Purpose
`run_llm_agent.sh`	`Prototyper/bin/run_llm_agent.sh`	**The LLM-call primitive.** Every script that invokes a model goes through it — never `claude` directly. Wraps the CLI agent (Claude by default; Codex via `USE_CODEX=1` / `LLM_PROVIDER=codex`). Prompt on stdin; final text to stdout; events and token usage to `$RAW_LOG` / `$LOG_FILE`. `--stream` runs the streaming pipeline. The single seam through which all batched AI calls pass.
`build_documentation.sh`	`Prototyper/bin/build_documentation.sh`	**The only documentation command ever called.** Builds project documentation, exposed via `docs/index.html`, and cascades to the white-paper renderer when `docs/whitepapers/*.md` exist. All sub-builders are convenience methods, not invoked directly.
`build_pdf.sh`	`Prototyper/bin/build_pdf.sh <file>`	Convert a `.md` (branded white paper) or `.html` file to a PDF in the **current directory**. Extension-driven: `.md` → HTML → PDF, `.html` → PDF. Renders via headless Chromium.
newwin	`Developer-Tooling/newwin`	Opens a 3-pane IDE tab in Windows Terminal for a project. Always opens in a named window `BASE` — creates it on first call, adds a tab thereafter. Each project is locked to one of 10 colour schemes. Requires `$MYPROJECTS`; state in `newwin.json`.
log_entry	`Developer-Tooling/log_entry.py`	Platform LEARNINGS log helper; backs the `slearn` / `stodo` /... shell functions.

Shell Setup

`bash` configured in `~/.bashrc` (sourced by `~/.profile` for login shells). Backups are kept dated: `~/.bashrc.YYYYMMDD`.

Conventions

- `set -o vi` — vi editing mode on the command line; `EDITOR` / `VISUAL` set to `vi` to match.
- **Shared history across all terminals**

```
shopt -s histappend
HISTSIZE=100000
HISTFILESIZE=200000
```

```
HISTTIMEFORMAT='%F %T  '
PROMPT_COMMAND="history -a; history -c; history -r${PROMPT_COMMAND:+; $PROMPT_COMMAND}"
```

- **PATH** — adds `$MYPROJECTS/Developer-Tooling`, `~/bin`, and `~/.local/bin`. `nvm` loads Node.

Main aliases and functions

Alias / function	Action
`pro`	`cd $MYPROJECTS`
`myidea` / `myquery` / `mytrack`	`book.py idea` / `query` / `track`
`mntg`	Mount `G:` at `/mnt/g`
`ollama`	Windows Ollama binary via WSL interop
`slearn` `stodo` `sbook` `sidea` `scomplete`	Append an entry to the platform `LEARNINGS.md` via `log_entry.py`
`fd` / `bat`	Aliased to Ubuntu's `fdfind` / `batcat`

Utilities

Command-line utilities on the box. Description in black, **install command in blue**, usage in monospace.

fzf — fuzzy finder.
`sudo apt-get install -y fzf`
Usage: `Ctrl-R` fuzzy history · `Ctrl-T` file picker · `Alt-C` fuzzy cd.
Configured — key bindings sourced in .bashrc.

fd — fast, ergonomic `find` replacement.
`sudo apt-get install -y fd-find`
Usage: `fd PATTERN` · `fd -e py` by extension · `fd -H` include hidden.
Configured — aliased fd → fdfind in .bashrc.

bat — `cat` with syntax highlighting, line numbers, and a git gutter.
`sudo apt-get install -y bat`
Usage: `bat FILE` · `bat -n` line numbers · `bat -p` plain.
Configured — aliased bat → batcat in .bashrc.

ncdu — interactive disk-usage browser; find what is eating space.
`sudo apt-get install -y ncdu`
Usage: `ncdu DIR`; arrows navigate, `d` delete, `q` quit.

entr — run a command when files change.
`sudo apt-get install -y entr`
Usage: `ls *.py | entr -r ./bin/test.sh` (`-r` reload, `-c` clear).

direnv — per-directory environment / auto venv activation on `cd`.
`sudo apt-get install -y direnv`

Usage: write `.envrc`, then `direnv allow`.
Further work — add `eval "$(direnv hook bash)"` to .bashrc.

lazygit — terminal git UI: staging, branches, history.
`sudo snap install lazygit`
Usage: `lazygit` in a repo; `space` stage, `c` commit, `P` push.
Not installed.

btop — rich resource monitor (CPU / GPU / memory).
`sudo snap install btop`
Usage: `btop`; `m` menu, `q` quit.
Not installed.

yq — `jq` for YAML / TOML.
`uv tool install yq`
Usage: `yq '.key' file.yaml`.
Not installed.

shfmt — shell formatter; pairs with `shellcheck` for `bin/*.sh`.
`go install mvdan.cc/sh/v3/cmd/shfmt@latest`
Usage: `shfmt -w bin/script.sh`.
Not installed (Go).

To enumerate what is already installed when refreshing this list:

```
apt-mark showmanual | sort                # explicitly installed apt packages
snap list                                 # snap packages
uv tool list                              # python CLI tools
npm -g ls --depth=0                       # node globals
ls "/mnt/c/Users/barlo/AppData/Local/Programs"   # Windows per-user installs
```

Agent Behaviour

`~/.claude/CLAUDE.md` and `~/.codex/AGENTS.md` hold the agent contract. Summary:

Git policy

- **Commit immediately** after completing any task that has no errors.
- Commit messages are descriptive — **no "OpenAi", "Claude", "Anthropic", or "AI" mentions**.
- **Do not push.** Local commits only.
- **No co-authored-by lines.**

Model invocation

- **Avoid paid API-token usage** — no metered API keys in scripts or tools.
- **Route model calls through the `run_llm_agent.sh` primitive** as much as possible. It runs via the CLI subscription and honours the `USE_CODEX` environment variable.

Directory scope

- Projects live under `$MYPROJECTS`; files therein are read/write. No access outside that tree without explicit permission.

Response voice (BRANDING_EDSVOICE)

How LLMs respond in interactive sessions — `RulesEngine/BRANDING_EDSVOICE.md`:

- **Lead with the answer or decision**, then the why, then the key tradeoff. No preamble.
- Terse and professional; no hedging; own the recommendation.
- **Formal English in written artifacts** — no contractions ("you are", "do not").
- Use the word **specification**, not "spec" (except in code itself).
- End every response with the terminator: `---------- REQUEST COMPLETED ----------`.
- Before the terminator, include `NEXT STEPS` (actions / blocked work) and `QUESTIONS` (decisions only Ed can make). Carry unresolved items forward.

Project Standards

These apply per project.

Standard	Rule
Conformed Project	A project brought to platform standards — carries a `CLAUDE_RULES.md` block in `AGENTS.md` and a `METADATA.md` as authoritative identity.
Python environment	Anaconda on the Windows side; `uv` for virtual environments and dependencies in WSL.
Lint / format	`ruff` for linting and formatting.
Testing	`pytest`; `bin/test.sh` runs the suite and must pass before commit.
Scripts	Executables live in `bin/` (generally bash); a `# CommandCenter Operation` header registers them with the platform.
Shared libs	`common.sh` / `common.py` provide env loading, logging, SIGTERM, heartbeat. Source / import rather than re-implement.
Line endings	Linux (no `\r`); `chmod +x bin/*.sh`.

A **Conformed Project** carries a generated `CLAUDE_RULES.md` block in its `AGENTS.md` (markers `CLAUDE_RULES_START` … `CLAUDE_RULES_END`), produced from `RulesEngine/BUSINESS_RULES.md` and injected by Prototyper tooling — replaceable, do not hand-edit it. Its `METADATA.md` is authoritative for `name`, `display_name`, `short_description`, `git_repo`, never inferred from directory names. Conformance is mechanised by `bin/ProjectInitialize.sh` (assess → mechanical fixes → agent conformance → validate); it is idempotent.

Standard Directory Layout

Code directories carry no `.py` or `.sh` programs in their root.

```
<project>/
  bin/          code execution (start.sh, test.sh, …)
  src/          source
  docs/         documentation
  data/         data
```

```
inbound/      inbound raw data
Prompts/      prompts
```

Documentation and Voices

`build_documentation.sh` is the single documentation entry point; the **documentation agent** in Prototyper writes project documentation from existing docs, code, or specifications.

Voice is tooling: each **output** carries its own register, independent of the others. The target is either **me** or a **program/output you write** — never a third-party recipient. All inherit the brand from `RulesEngine/BRANDING_MAIN.md`.

Output	Voice file
Chat to Ed	`RulesEngine/BRANDING_EDSVOICE.md`
Documentation site	`RulesEngine/BRANDING_DOCUMENTATION.md`
White papers	`RulesEngine/BRANDING_WHITEPAPERS.md`
Website / portfolio	`RulesEngine/BRANDING_WEBSITE.md`
Social / blog posts	`RulesEngine/BRANDING_POSTS.md`

Not yet defined: proposals (client-facing) — to be authored.

Skills

Global skills live in `~/.claude/skills/`; project skills in each project's `.claude/skills/`.

Skill	Scope	Purpose
`make_plan`	Global	Produce an implementation plan before coding.
`live-iterate`	Prototyper	In-session editing of a specification and its code — the BOTH behaviour without a cold run.
`update-rules`	Prototyper	Edit the rules engine and regenerate `CLAUDE_RULES.md`.

The former global `iterate` skill is retired; use `live-iterate`.